

LLMのインタープリター・モードとコンパイル・モード

— 探索はLLMに、確立した手順は決定的なコードに。bitbank-lab-cli による実証 —

著者: aobathree (ChatGPT・Claude・Cursor の助けを借りて作成) | 初版 2026年8月1日 / 改訂版 2026年8月2日 (Rust実験を追記)
リポジトリ: <https://github.com/aobathree/llm-interpreter-compiler-modes> (本稿・ドラフト・実証コードを収録)

はじめに：こんなアイデアは成り立つでしょうか

同じAPI^[1]の上に、MCP^[2]とCLI^[3]という二つの入口があるとします。初回はMCPを使い、LLMに多くのトークン^[4]と時間をかけて調査・計画・実行をしてもらう。これを**インタープリター・モード**と呼びます。2回目以降は、確立した同じ手順を「コンパイル」して、CLIを使ったスクリプトとして高速・低コストで再実行する。これが**コンパイル・モード**です。

【インタープリター・モード（初回）】

ユーザー → LLM → MCP → API
↑ 試行錯誤しながら手順を発見
(柔軟。ただし遅く、トークン消費が大きい)

【コンパイル・モード（2回目以降）】

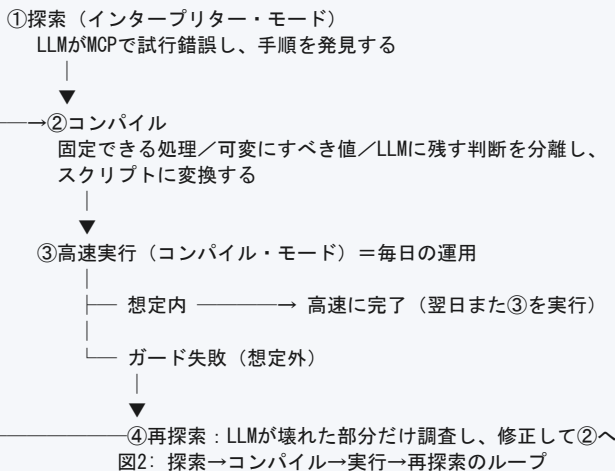
ユーザー → スクリプト → CLI → API
↑ 発見済みの手順をそのまま実行
(高速・安価。毎回同じ結果)
図1: 二つのモード

このアイデアは成り立つのか、それとも既に当たり前なのか——ChatGPTとCursorに問いかけた回答と、bitbank-lab-cliでの実例をまとめたのが本稿です。

概念の整理：「探索」と「実行」を分ける

両者の回答は一致していました。このアイデアは成り立ち、業界もまさにその方向へ収束しつつある、というものです。

本質は、「未知の問題の手順を**発見**するフェーズ」と「発見済みの手順を**再実行**するフェーズ」の分離にあります。前者は柔軟ですが遅く高価、後者は高速・安価で、毎回同じ結果が得られます。さらに、想定外が起きたときだけLLMに戻る「再探索」を加えると、全体は次のようなループになります。



これは、事前に全部を機械語にってしまうAOTコンパイル^[5]というより、実際の実行の様子を観察してから最適化する**JITコンパイル**^[6]に近い構図です。想定外の際に一段遅いモードへ戻る④は、JITでいう「脱最適化」^[7]に相当します。

この方向性は業界の潮流とも一致します。CloudflareのCode ModeやAnthropicの「Code execution with MCP」は「MCPツールを一つずつLLMに呼ばせるのではなく、コードを書かせて実行するほうが速くて安い」という同じ問題意識を扱っていますし、Agent Skills^[8]は確立した手順の資産化にあたります。研究の世界でもVoyagerやDSPyなど先行例の系譜があります。ただし、図2のループ全体——フォールバックと再コンパイルまで含めたライフサイクル——を一貫した体験として設計した例はまだ少なく、体系化の余地が大きい領域です。

何をコンパイルできるのか

手順のすべてをスクリプトにできるわけではありません。LLM の実行履歴は、次の三種類に分解して扱う必要があります。

LLMの実行履歴

- ① 完全に固定できる処理 → 純粋なコードへ
(API取得・集計・形式変換・保存)
- ② 可変にすべき値 → 入力変数へ
(前回の「BTC」を \${pair} に置き換える)
- ③ 意味的な判断が残る処理 → 小さなLLM呼び出しとして残す
(「重要なものだけ選ぶ」「異常なら深掘りする」)

図3: コンパイル時の三分解

つまり最適な結果は「LLM を完全に消したスクリプト」とは限らず、多くの場合は「**決定論的な骨格 + 少数の限定された LLM 判断 + 異常時の LLM 復帰**」という構成になります。また、単純な「記録して再生」では、たまたま必要だったリトライの誤固定や、値のハードコード、書き込みの二重実行などで壊れます。可変値の識別、冪等性^[9]への配慮、想定外を検知するガード条件、コンパイル直後に同じ入力で結果一致を確かめるゴールドンテスト^[10]が最低限必要です。

■ 実例：bitbank-lab-cli の daily_brief.py

このモデルを、暗号通貨の日次「商い状況」分析で実際に運用しています。

インタープリター・モード（初回）では、bitbank candles で数百本×複数ペアの生のローソク足^[11] JSONを取得し、LLM がそれを読み込んで SMA や RSI などのテクニカル指標^[12] を手計算していました。生JSONだけで数万トークンを消費し、計算は遅く、桁ミスの検算コストもかかります。ただ、この試行錯誤を通じて「毎日見るべき指標と出力の形」が確定しました。

コンパイル・モード（2回目以降）では、確定した手順を `tools/daily_brief.py` に固定化しました。計算はすべて Python 側で行い、LLM は1ペア3行の圧縮ダイジェストだけを読みます。

【従来】 bitbank → 生JSON（数万トークン） → LLMが読む＆手計算 → 要約
ここが重い

【改善】 bitbank → daily_brief.py が計算 → ダイジェスト10行 → LLMは10行だけ読む
計算0.44秒 軽い！

図4: daily brief.py によるトークン削減

スクリプト本体と設計ノートは上記リポジトリの `tools/` に収録しています。Python 3.9+ と `bitbank-lab-cli` (APIキー不要) があれば、`python tools/daily_brief.py` だけでこの実験をそのまま再現できます。

実測の効果は、実行時間**約0.44秒**（2ペア）、LLM が読む量は**数万トークン → 10行弱**、指標の手計算・検算は**ゼロ**です。取得本数を増やしても LLM のトークン消費は一定のままです。図3の三分解も実践しています——対象ペアは引数や環境変数で指定できるようにし ②、確定足のみで計算する・ペア単位でエラーを閉じ込めるといったガードを備え ①、「ダイジェストへの一言コメント」だけを LLM に残しています ③。運用の合言葉は「daily brief 回して」。LLM はスクリプトを実行し、出てきた10行だけを読んでコメントを添えます。

コンパイルを推し進める：Rust移植の実験（8/2追記）

コンパイル・モードはどこまで速くできるのでしょうか。daily_brief.py を Rust^[13] に移植して（リポジトリの rust/ に収録）、実測してみました。出力フォーマットはPython版と1文字単位で同一です。変えたのは実行モデルだけ——ネイティブバイナリ化と、fetch（3ペア×日足・時足＝6回のCLI呼び出し）の全並列化^[14] です。

【第0段】インタープリター・モード
LLMが生JSONを読んで手計算 …… 数万トークン・分オーダー
↓
手順をスクリプトに固定（コンパイル）

【第1段】Python版 daily_brief.py …… 約0.6秒
↓
ネイティブ化+fetch並列化（さらにコンパイル）

【第2段】Rust版 …… 約0.11秒
図5：コンパイルの深化と実測値

実装	実行時間（3ペア・中央値）	備考
Python版	約0.6秒	6回のfetchが逐次
Rust版	約0.11秒（約5倍）	6回のfetchを並列化 → 実質1回分の待ち時間

計測して分かったのは、律速がネットワークだということです。単発の bitbank candles 呼び出しは約0.1秒で、Python版はこれを6回順番に待つため約0.6秒かかります。Rust版は6回を同時に投げるので、待ち時間が実質1回分に圧縮されます。指標計算（SMA/RSI/MACD/ATR）そのものはどちらの実装でも数ミリ秒以下で、差にはなりません。

ここから得られる教訓は「Rustだから速い」ではありません。手順が決定的なコードに固定されている——つまりコンパイル済みである——からこそ、並列化という古典的な最適化を安全に適用できた、という点です。もしLLMの判断が実行パスに挟まっていたら、「6回の呼び出しを同時に投げてよいか」をそのつど推論することになります。数万トークン→10行という第0段→第1段の跳躍に比べれば地味ですが、この実験は「コンパイルには深さの段階がある」ことを示しています。

まとめ

「LLM は初回の探索と手順発見に使い、確立した手順は決定的なコードに落として再利用する」という二相モデルは成り立ち、業界もその方向へ向かっています。価値の中心は個々の技術ではなく、探索 → 記録 → 三分解 → コンパイル → 高速実行 → 想定外だけ LLM へ復帰 → 再コンパイルというループ（図2）を一つの体験として統合することにあります。daily_brief.py はその最小の実装です。次の課題は、相関分析やボラティリティ分析など、同じ型の横展開です。

用語注釈

- API**: ソフトウェア同士が機能呼び出し合うための窓口。ここでは bitbank の価格データ取得サービスなどを指します。
- MCP** (Model Context Protocol): LLM が外部のツールやデータを呼び出すための標準的な接続方式。LLM が「自分で道具を選んで使う」ための仕組みです。
- CLI** (Command Line Interface): コマンド（文字列の命令）でソフトウェアを操作する方式。人にもスクリプトにも同じように呼び出せるのが特長です。
- トークン**: LLM が文章を処理する最小単位。日本語なら概ね1～2文字で1トークン。LLM の利用料金と処理時間はトークン量に比例します。
- AOTコンパイル** (Ahead-Of-Time): プログラムを実行前にあらかじめ全部機械語に変換しておく方式。
- JITコンパイル** (Just-In-Time): プログラムを実行しながら、よく使う部分だけをその場で機械語に変換して高速化する方式。実際の動きを観察してから最適化する点が本稿のアイデアに似ています。
- 脱最適化** (deoptimization): JIT で高速化した部分が前提と合わなくなったとき、いったん遅いが確実な実行方式へ戻すこと。
- Agent Skills**: Claude Code や Cursor など、確立した作業手順を手順書ファイルとして保存し、次回以降に再利用する仕組み。
- 冪等性** (べきとうせい): 同じ処理を何回実行しても結果が1回のとときと変わらない性質。二重登録などの事故を防ぐ鍵になります。
- ゴールデンテスト**: 正解とみなす出力（ゴールデン）をあらかじめ保存しておき、変更後も同じ入力から同じ出力が得られるか照合するテスト手法。
- ローソク足**: 一定期間の始値・高値・安値・終値をまとめた、相場チャートの標準的なデータ形式。
- テクニカル指標** (SMA・RSI・MACD・ATR): 価格の推移から計算する分析値。SMA は移動平均、RSI は買われすぎ・売られすぎの目安、MACD はトレンドの勢い、ATR は変動幅の大きさを表します。
- Rust**: 実行前に機械語へ変換される（コンパイルされる）プログラミング言語。起動が速く、複数の処理を同時に走らせる並列実行を安全に書けることで知られます。
- 並列化（並列実行）**: 複数の処理を同時に走らせること。本稿では6回のデータ取得を同時に行い、待ち時間を1回分に圧縮しました。